

The pre-trained model that we have tried during the experiments:

Model	Hidden Size	Number of Layers	Parameters	Speed	Memory
<code>bert-base-uncased</code>	768	12	110M	Slow	High
<code>distilbert-base-uncased</code>	768	6	66M	Fast	Moderate
<code>albert-base-v2</code>	768	12	12M	Fast	Low
<code>huawei-noah/TinyBERT</code>	312	4	~15M	Very Fast	Very Low
<code>distilroberta-base</code>	768	6	82M	Fast	Moderate

The process:

Our dataset consists of 416,809 records, and when we attempted to fine-tune the `bert-base-uncased` pre-trained model, it required an extensive runtime, often causing the system to crash. The complexity of the `bert-base-uncased` model itself further added to the challenge, making it unnecessary and inefficient to train directly on it. To address this, we decided to use pre-trained models to extract latent features, which we could then use to train our own neural networks.

However, even with this approach, the process was still time-consuming and resource-intensive. We experimented with several simpler transformer-based models, including `distilbert-base-uncased`, `albert-base-v2`, and `tinybert`. We specifically chose transformer-based models because they retain positional information, which is crucial for our task. Ultimately, we found a viable solution by using only a subset of the dataset for training and opting for a relatively simpler transformer-based model: the `huawei-noah/TinyBERT`.

